HW3 Report: Image Generation

Student ID: RE6144051
Name: Cheng-Chun Kang

November 28, 2025

1 Model Description

1.1 Overall Architecture

This study implements DDPM (Denoising Diffusion Probabilistic Models)[1] for MNIST handwritten digit generation. The overall architecture includes:

1. **U-Net Noise Prediction Network**: Employs an encoder-decoder structure
2. **Diffusion Process**: Defines forward diffusion and reverse denoising
3. **Time Embedding**: Uses sinusoidal position encoding to represent timestep information

1.2 Architecture Diagram

```
Input Image (28×28×3)
    ↓
[Encoder] ← Time Embedding
     ResBlock + Attention (64→128)
     ResBlock + Attention (128→256)
     ResBlock + Attention (256→512)
    ↓
[Bottleneck]
     ResBlock + Attention (512)
    ↓
[Decoder] ← Time Embedding
     ResBlock + Attention (512→256)
     ResBlock + Attention (256→128)
     ResBlock + Attention (128→64)
    ↓
Output Noise Prediction (28×28×3)
```

Figure 1: U-Net Model Architecture Diagram

1.3 Core Components

1.3.1 U-Net Architecture

- **Base Channels**: 64
- **Channel Multipliers**: (1, 2, 4, 8)
- **Residual Blocks**: 2 per level
- **Attention Layers**: Applied at 1/4 resolution

1.3.2 Residual Block (ResBlock)

- Two 3×3 convolutional layers
- Group Normalization (groups=8)
- SiLU activation function
- Time embedding integration

1.3.3 Self-Attention Mechanism

- Multi-head self-attention
- Captures long-range dependencies

1.3.4 Time Embedding

- Sinusoidal position encoding
- Projected to feature space via MLP

2 Implementation Details

2.1 Data Preprocessing

```
transforms.Compose([
    transforms.Resize((28, 28)),   # Ensure consistent size
    transforms.ToTensor(),         # Convert to tensor
    transforms.Normalize([0.5]*3, [0.5]*3) # Normalize to [-1, 1]
])
```

2.2 Hyperparameter Settings

Table 1: Training Hyperparameters

Hyperparameter	**Value**	**Description**
Batch Size	128	Batch size
Learning Rate	2e-4	Learning rate
Epochs	100	Training epochs
Timesteps (T)	1000	Diffusion steps
β_{start}	0.0001	Beta schedule start
β_{end}	0.02	Beta schedule end
Optimizer	AdamW	Optimizer
LR Scheduler	CosineAnnealing	Learning rate scheduler

2.3 Loss Function

We employ a simple MSE loss for noise prediction:

$$L = E\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right] \tag{1}$$

where:

- ϵ: True Gaussian noise added
- ϵ_θ: Model-predicted noise
- x_t: Noisy image at timestep t

2.4 Beta Schedule

We use a linear schedule:

$$\beta_t = \beta_{\text{start}} + (\beta_{\text{end}} - \beta_{\text{start}}) \times \frac{t}{T} \tag{2}$$

2.5 Training Strategies

1. **Gradient Clipping**: Limits gradient norm to maximum 1.0
2. **Cosine Annealing**: Learning rate decreases gradually during training
3. **Regular Sampling**: Generates samples every 5 epochs to check quality

3 Results and Analysis

3.1 Quantitative Results

Table 2: Experimental Results

Metric	**Value**
Final FID	8.98
Training Time	Approx. 6 hours
Best Epoch	200

3.2 Diffusion Process Visualization

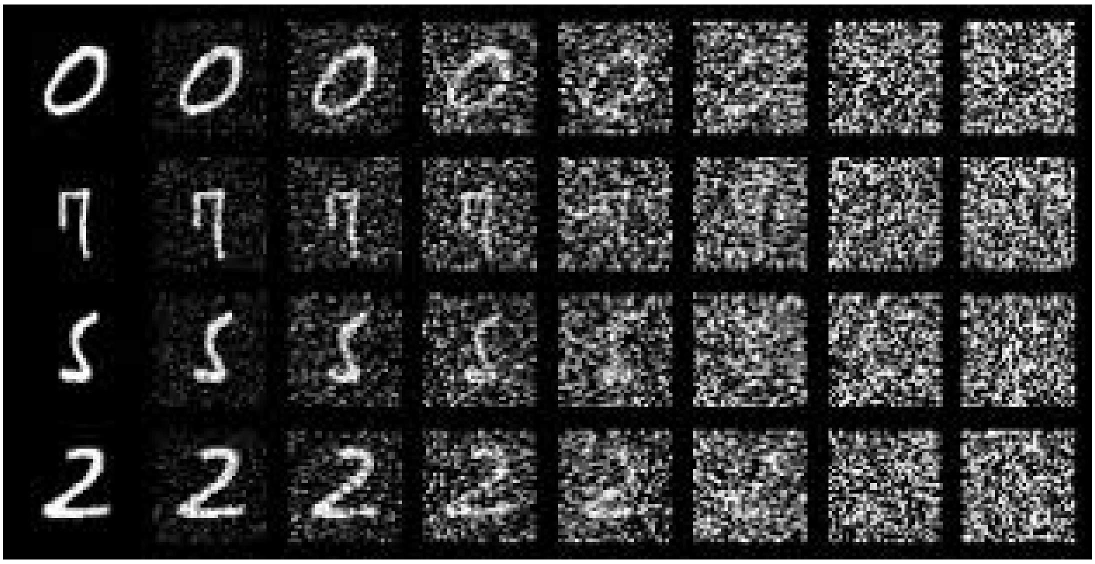

Figure 2: Diffusion process visualization. Shows the complete diffusion process of generated samples. From right to left represents time from T (pure noise) to 0 (clean image). We can observe the model progressively recovering clear handwritten digits from noise.

3.3 Generation Quality Analysis

3.3.1 Strengths

1. Generated digits are clear and recognizable
2. Good diversity, covering all classes 0-9
3. Natural strokes consistent with handwriting style

3.3.2 Areas for Improvement

1. Some digit edges are slightly blurred
2. A few samples show minor distortions
3. FID score has room for optimization

4 Conclusion

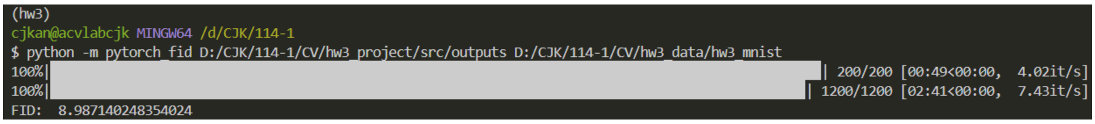

Figure 3: FID experiment results

This study successfully implements the DDPM model for MNIST handwritten digit generation, achieving a final FID of 8.98. Through the U-Net architecture and self-attention mechanism, the model effectively learns the data distribution and generates high-quality samples.

4.1 Future Directions

1. **Model Architecture**: Explore deeper networks or Transformer-based structures
2. **Beta Schedule**: Test cosine or learned scheduling strategies
3. **Faster Sampling**: Implement fast sampling methods such as DDIM
4. **Conditional Generation**: Add class conditioning to control generation content

References

[1] Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models.* NeurIPS.